



FIFO PROJECT



OSAMA MOHAMMED
SULTAN

-Verification plan:-

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
RESET_CHK	When the reset is asserted, the outputs' values should be low and the fifo will be empty	Directed at the start of the simulation, then randomized during simulation with constraints to drive the reset be off most of the time	-	immediate assertion to check for the async reset functionality + checker comparing the FIFO data with a golden model
WR_EN_CHK	when the wr_en is asserted and the fifo isn't full the data_in should be written in the fifo	randomized with the constraint to be on 70% of time during half of simulation then randomized with constraint to be off 70% of time during the other half	cover all combinations with the output flags (full, almostfull, empty, almost empty, overflow, underflow)	checker comparing the fifo's data with a golden model
RD_EN_CHK	when the rd_en is asserted and the fifo isn't empty the data_out should be equals to the first element in the fifo	randomized with the constraint to be on 30% of time during half of simulation then randomized with constraint to be off 30% of time during the other half	cover all combinations with the output flags (full, almostfull, empty, almost empty, overflow, underflow)	checker comparing the fifo's data with a golden model
FULL_CHK	when the fifo is full the full signal should be asserted	-	cover all combinations with the wr_en & rd_en signals	concurrent assertion to check the full_flag functionality on the same cycle
ALMOSTFULL_CHK	when there is only one space empty in the fifo the almostfull flag should be asserted	-	cover all combinations with the wr_en & rd_en signals	concurrent assertion to check the full_flag functionality on the same cycle
EMPTY_CHK	when the fifo is empty the empty flag should be asserted	-	cover all combinations with the wr_en & rd_en signals	concurrent assertion to check the full_flag functionality on the same cycle
ALMOSTEMPTY_CHK	when the there is only one data remaining in the fifo the almostempty flag should be asserted	-	cover all combinations with the wr_en & rd_en signals	concurrent assertion to check the full_flag functionality on the same cycle
OVERFLOW_CHK	when the wr_en is asserted and fifo is full the write operation should be rejected and the overflow flag should be asserted	-	cover all combinations with the wr_en & rd_en signals	concurrent assertion to check the full_flag functionality on the next positive edge clock + checker comparing the FIFO data with a golden model
UNDERFLOW_CHK	when the rd_en is asserted and fifo is empty the read operation should be rejected and the underflow flag should be asserted	-	cover all combinations with the wr_en & rd_en signals	concurrent assertion to check the full_flag functionality on the next positive edge clock + checker comparing the FIFO data with a golden model
WR_ACK_CHK	when the wr_en is asserted and the fifo isn't full the data_in should be written in the fifo and the wr_ack should be asserted	-	-	concurrent assertion tha checks the functionality of wr_ack on each positive clk cycle

-Bugs found:-

- Bug 1 :

- “data_out” signal didn’t go low when reset is asserted.

- correctness :

```
39  always @(posedge ifc.clk or negedge ifc.rst_n) begin
40      → if (!ifc.rst_n) begin
41          → rd_ptr <= 0;
42          → // data_out signals didn't reset
43          → ifc.data_out <= 0;
44          → end
```

- Bug 2 :

- “underflow” signal should be driven sequentially not combinational.

- correctness :

```
46 → → if (ifc.rd_en && count != 0) begin
47 → →   ifc.data_out <= mem[rd_ptr];
48 → →   rd_ptr <= rd_ptr + 1;
49 → →   ifc.underflow <= 0; // bug
50 → → end
51 → → else if (ifc.rd_en && count == 0) begin
52 → →   ifc.data_out <= 0;
53 → →   ifc.underflow <= 1; // bug
54 → → end
```

- Bug 3 :

- “almostfull” signal should be asserted when there is only one empty space left not two.

- correctness :

```
82 assign ifc.almostfull = (count == FIFO_DEPTH-1)? 1 : 0;
```

- Bug 4 :

- Didn't handle writing and reading simultaneously.

- correctness :

```
67 → → //error didn't handle both writing and reading case
68 ✓ → → else if (({ifc.wr_en, ifc.rd_en} == 2'b11)) begin
69 ✓ → →   if (ifc.empty) begin
70 → →     count <= (count + 1);
71 → →   end
72 ✓ → →   else if (ifc.full) begin
73 → →     count <= (count - 1);
74 → →   end
75 → → end
```

-Assertion coverage:-

Assertions													
File Edit View Add Bookmarks Window Help													
Assertions													
Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
/top/RESET_OUT_CHK	Immediate	SVA	on	0	1	-	-	-	-	0	off	assert (fc.data_out==0)	✓
/top/DUT/WR_ACK_CHK	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) disable l...	✓
/top/DUT/OVERFLOW_CHK	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) disable l...	✓
/top/DUT/UNDERFLOW_CHK	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) disable l...	✓
/top/DUT/FULL_CHK	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) (count=...	✓
/top/DUT/EMPTY_CHK	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) (count=...	✓
/top/DUT/ALMOST_FULL_CHK	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) (count=...	✓
/top/DUT/ALMOST_EMPTY_CHK	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) (count=...	✓
/top/DUT/WR_WRAP	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) disable l...	✓
/top/DUT/RD_WRAP	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) disable l...	✓
/top/DUT/THRESHOLD	Concurrent	SVA	on	0	1	-	0B	0B	0 ns	0	off	assert (@posedge ifc.clk) ((wr_ptr...	✓
/top/DUT/RESET_CHK/ASNYC_RST	Immediate	SVA	on	0	1	-	-	-	-	0	off	assert (wr_ptr==0&rd_ptr==0&co...	✓
/top/tb/publk#182146786#14/immed_...	Immediate	SVA	on	0	1	-	-	-	-	0	off	assert (randomize(...))	✓
/top/tb/publk#182146786#25/immed_...	Immediate	SVA	on	0	1	-	-	-	-	0	off	assert (randomize(...))	✓

Name	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph	Included	Memory	Peak Memory	Peak Memory Time	Cumulative Threads
/top/DUT/cover_pointer_threshold	SVA	✓	Off	20003	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__rd_wrwrap_p	SVA	✓	Off	440	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__wr_wrwrap_p	SVA	✓	Off	705	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__a_empty_p	SVA	✓	Off	4785	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__a_full_p	SVA	✓	Off	1644	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__empty_p	SVA	✓	Off	6761	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__full_p	SVA	✓	Off	2373	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__underflow_p	SVA	✓	Off	4115	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__overflow_p	SVA	✓	Off	1622	1	Unli...	1	100%	✓	✓	0	0	0 ns	0
/top/DUT/cover__wr_ack_p	SVA	✓	Off	8464	1	Unli...	1	100%	✓	✓	0	0	0 ns	0

-Functional coverage:-

Covergroups													
File Bookmarks Window Help													
Covergroups													
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment				
/FIFO_cov_plg/FIFO_cov		100.00%											
TYPE cov		100.00%	100	100.00...	✓				auto(1)				
CVP cov:WR_EN_COV		100.00%	100	100.00...	✓								
CVP cov:RD_EN_COV		100.00%	100	100.00...	✓								
CVP cov:FULL_COV		100.00%	100	100.00...	✓								
CVP cov:A_FULL_COV		100.00%	100	100.00...	✓								
CVP cov:EMPTY_COV		100.00%	100	100.00...	✓								
CVP cov:A_EMPTY_COV		100.00%	100	100.00...	✓								
CVP cov:OVERFLOW_COV		100.00%	100	100.00...	✓								
CVP cov:UNDERFLOW_COV		100.00%	100	100.00...	✓								
CVP cov:WR_ACK_COV		100.00%	100	100.00...	✓								
CROSS cov:FULL_CROSS		100.00%	100	100.00...	✓								
bin <wr_lowrd_low_full_low>		2741	1	100.00...	✓								
bin <wr_lowrd_low_full_high>		458	1	100.00...	✓								
bin <wr_highrd_low_full_low>		3737	1	100.00...	✓								
bin <wr_highrd_low_full_high>		2041	1	100.00...	✓								
bin <wr_lowrd_high_full_low>		5680	1	100.00...	✓								
bin <wr_highrd_high_full_low>		5346	1	100.00...	✓								
ignore_bin cant_occur		0	-	-	✓								
CROSS cov:A_FULL_CROSS		100.00%	100	100.00...	✓								
bin <wr_lowrd_low_a_full_low>		2854	1	100.00...	✓								
bin <wr_highrd_low_a_full_low>		5419	1	100.00...	✓								
bin <wr_lowrd_high_a_full_low>		5478	1	100.00...	✓								
bin <wr_highrd_high_a_full_low>		4518	1	100.00...	✓								
bin <wr_lowrd_low_a_full_high>		245	1	100.00...	✓								
bin <wr_highrd_low_a_full_high>		359	1	100.00...	✓								
bin <wr_lowrd_high_a_full_high>		202	1	100.00...	✓								
bin <wr_highrd_high_a_full_high>		828	1	100.00...	✓								
CROSS cov:A_EMPTY_CROSS		100.00%	100	100.00...	✓								
bin <wr_lowrd_low_a_empty_low>		2519	1	100.00...	✓								
bin <wr_highrd_low_a_empty_low>		4988	1	100.00...	✓								
bin <wr_lowrd_high_a_empty_low>		5336	1	100.00...	✓								
bin <wr_highrd_high_a_empty_low>		2135	1	100.00...	✓								
bin <wr_lowrd_low_a_empty_high>		680	1	100.00...	✓								
bin <wr_highrd_low_a_empty_high>		790	1	100.00...	✓								
bin <wr_lowrd_high_a_empty_high>		344	1	100.00...	✓								
bin <wr_highrd_high_a_empty_high>		3211	1	100.00...	✓								

CROSS cvg:EMPTY_CROSS	100.00%	100	100.00%	✓
bin <wr_low,rd_low,empty_low>	2334	1	100.00%	✓
bin <wr_high,rd_low,empty_low>	5506	1	100.00%	✓
bin <wr_low,rd_high,empty_low>	990	1	100.00%	✓
bin <wr_high,rd_high,empty_low>	5093	1	100.00%	✓
bin <wr_low,rd_low,empty_high>	865	1	100.00%	✓
bin <wr_high,rd_low,empty_high>	272	1	100.00%	✓
bin <wr_low,rd_high,empty_high>	4690	1	100.00%	✓
bin <wr_high,rd_high,empty_high>	253	1	100.00%	✓
CROSS cvg:OVERFLOW_CROSS	100.00%	100	100.00%	✓
bin <wr_low,rd_low,overflow_low>	3173	1	100.00%	✓
bin <wr_high,rd_low,overflow_low>	4064	1	100.00%	✓
bin <wr_low,rd_high,overflow_low>	5673	1	100.00%	✓
bin <wr_high,rd_high,overflow_low>	4602	1	100.00%	✓
bin <wr_low,rd_low,overflow_high>	25	1	100.00%	✓
bin <wr_high,rd_low,overflow_high>	1714	1	100.00%	✓
bin <wr_low,rd_high,overflow_high>	7	1	100.00%	✓
bin <wr_high,rd_high,overflow_high>	744	1	100.00%	✓
CROSS cvg:UNDERFLOW_CROSS	100.00%	100	100.00%	✓
bin <wr_low,rd_low,under_low>	2377	1	100.00%	✓
bin <wr_high,rd_low,under_low>	4925	1	100.00%	✓
bin <wr_low,rd_high,under_low>	3038	1	100.00%	✓
bin <wr_high,rd_high,under_low>	3469	1	100.00%	✓
bin <wr_low,rd_low,under_high>	820	1	100.00%	✓
bin <wr_high,rd_low,under_high>	853	1	100.00%	✓
bin <wr_low,rd_high,under_high>	2642	1	100.00%	✓
bin <wr_high,rd_high,under_high>	1877	1	100.00%	✓
CROSS cvg:WR_ACK_CROSS	100.00%	100	100.00%	✓
CROSS cvg:WR_ACK_CROSS	100.00%	100	100.00%	✓
bin <wr_low,rd_low,wr_ack_low>	3131	1	100.00%	✓
bin <wr_high,rd_low,wr_ack_low>	1346	1	100.00%	✓
bin <wr_low,rd_high,wr_ack_low>	5558	1	100.00%	✓
bin <wr_high,rd_high,wr_ack_low>	642	1	100.00%	✓
bin <wr_low,rd_low,wr_ack_high>	67	1	100.00%	✓
bin <wr_high,rd_low,wr_ack_high>	4432	1	100.00%	✓
bin <wr_low,rd_high,wr_ack_high>	122	1	100.00%	✓
bin <wr_high,rd_high,wr_ack_high>	4704	1	100.00%	✓

-code coverage:-

```

=====
=== Instance: /top/ifc
=== Design Unit: work.FIFO_ifc
=====
Toggle Coverage:
Enabled Coverage      Bins      Hits      Misses      Coverage
-----
Toggles              86        86         0      100.00%

=====Toggle Details=====
Toggle Coverage for instance /top/ifc --

Node      1H->0L      0L->1H      "Coverage"
-----
almostempty      1          1      100.00
almostfull       1          1      100.00
clk              1          1      100.00
data_in[15-0]    1          1      100.00
data_out[15-0]   1          1      100.00
empty            1          1      100.00
full             1          1      100.00
overflow         1          1      100.00
rd_en            1          1      100.00
rst_n            1          1      100.00
underflow        1          1      100.00
wr_ack           1          1      100.00
wr_en            1          1      100.00

Total Node Count    =      43
Toggled Node Count  =      43
Untoggled Node Count =       0

Toggle Coverage     =    100.00% (86 of 86 bins)

```

```

Branch Coverage:
Enabled Coverage      Bins   Hits   Misses  Coverage
-----
Branches              28     28     0    100.00%

=====Branch Details=====

Branch Coverage for instance /top/DUT

Line      Item      Count   Source
----
File FIFO.sv
-----IF Branch-----
20          1      20903   Count coming in to IF
20          1      1842    if (l1fc.rst_n) begin
25          1      8887    else if (l1fc.wr_en && count < FIFO_DEPTH) begin
30          1      10174   else begin
Branch totals: 3 hits of 3 branches = 100.00%

-----IF Branch-----
32          1      10174   Count coming in to IF
32          1      1713    if (l1fc.full & l1fc.wr_en)
34          1      8461    else
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
40          1      20903   Count coming in to IF
40          1      1842    if (l1fc.rst_n) begin
46          1      6193    if (l1fc.rd_en && count != 0) begin
51          1      4312    else if (l1fc.rd_en && count == 0) begin
51          1      8556    All False Count
Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----
59          1      17957   Count coming in to IF
59          1      1826    if (l1fc.rst_n) begin
62          1      16131   else begin
Branch totals: 2 hits of 2 branches = 100.00%

```

```

-----IF Branch-----
63          1      16131   Count coming in to IF
63          1      4295    if (l1fc.wr_en, l1fc.rd_en) == 2'b10 && l1fc.full)
65          1      2888    else if (l1fc.wr_en, l1fc.rd_en) == 2'b01 && l1fc.empty)
68          1      4389    else if (l1fc.wr_en, l1fc.rd_en) == 2'b11) begin
68          1      4559    All False Count
Branch totals: 4 hits of 4 branches = 100.00%

-----IF Branch-----
69          1      4389   Count coming in to IF
69          1      1789    if (l1fc.empty) begin
72          1      502     else if (l1fc.full) begin
72          1      2098    All False Count
Branch totals: 3 hits of 3 branches = 100.00%

-----IF Branch-----
79          1      10155   Count coming in to IF
79          1      830     assign l1fc.full = (count == FIFO_DEPTH)? 1 : 0;
79          2      9325    assign l1fc.full = (count == FIFO_DEPTH)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
80          1      10155   Count coming in to IF
80          1      2579    assign l1fc.empty = (count == 0)? 1 : 0;
80          2      7576    assign l1fc.empty = (count == 0)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
82          1      10155   Count coming in to IF
82          1      1063    assign l1fc.almostfull = (count == FIFO_DEPTH-1)? 1 : 0; //==> bug almost full should assert when the fifo has only one empty space
82          2      9092    assign l1fc.almostfull = (count == FIFO_DEPTH-1)? 1 : 0; //==> bug almost full should assert when the fifo has only one empty space
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
83          1      10155   Count coming in to IF
83          1      2923    assign l1fc.almostempty = (count == 1)? 1 : 0;
83          2      7232    assign l1fc.almostempty = (count == 1)? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
88          1      14928   Count coming in to IF
88          1      1753    if (l1fc.rst_n) begin
88          1      13175   All False Count
Branch totals: 2 hits of 2 branches = 100.00%

```

Statement Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	27	27	0	100.00%
=====Statement Details=====				
Statement Coverage for instance /top/DUT --				
Line	Item	Count	Source	
----	----	-----	-----	
File FIFO.sv				
8			module FIFO(FIFO_ifc.dut ifc);	
9			parameter FIFO_WIDTH = 16;	
10			parameter FIFO_DEPTH = 8;	
11				
12			localparam max_fifo_addr = \$clog2(FIFO_DEPTH);	
13				
14			reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];	
15				
16			reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;	
17			reg [max_fifo_addr:0] count; //bug count didn't wraparound	
18				
19	1	20903	always @(posedge ifc.clk or negedge ifc.rst_n) begin	
20			if (!ifc.rst_n) begin	
21				
22	1	1842	wr_ptr <= 0;	
23				
24			end	
25			else if (ifc.wr_en && count < FIFO_DEPTH) begin	
26	1	8887	mem[wr_ptr] <= ifc.data_in;	
27	1	8887	ifc.wr_ack <= 1;	
28	1	8887	wr_ptr <= wr_ptr + 1;	
29			end	
30			else begin	
31	1	10174	ifc.wr_ack <= 0;	
32			if (ifc.full & ifc.wr_en)	
33	1	1713	ifc.overflow <= 1;	
34			else	
35	1	8461	ifc.overflow <= 0;	
36			end	
37			end	
38				
39	1	20903	always @(posedge ifc.clk or negedge ifc.rst_n) begin	
40			if (!ifc.rst_n) begin	
41	1	1842	rd_ptr <= 0;	
42			// data_out signals didn't reset	
43	1	1842	ifc.data_out <= 0;	
44			end	
45			else	
46			if (ifc.rd_en && count != 0) begin	
47	1	6193	ifc.data_out <= mem[rd_ptr];	
48	1	6193	rd_ptr <= rd_ptr + 1;	
49	1	6193	ifc.underflow <= 0; // bug	
50			end	
51			else if (ifc.rd_en && count == 0) begin	
52	1	4312	ifc.data_out <= 0;	
53	1	4312	ifc.underflow <= 1; // bug	
54			end	
55				
56			end	
57				
58	1	17957	always @(posedge ifc.clk or negedge ifc.rst_n) begin	
59			if (!ifc.rst_n) begin	
60	1	1826	count <= 0;	
61			end	
62			else begin	
63			if ((ifc.wr_en, ifc.rd_en) == 2'b10) && !ifc.full)	
64	1	4295	count <= (count + 1) % (FIFO_DEPTH + 1);	
65			else if ((ifc.wr_en, ifc.rd_en) == 2'b01) && !ifc.empty)	
66	1	2888	count <= (count - 1) % (FIFO_DEPTH + 1);	
67			//error didn't handle both writing and reading case	
68			else if (((ifc.wr_en, ifc.rd_en) == 2'b11)) begin	
69			if (!ifc.empty) begin	
70	1	1789	count <= (count + 1) % (FIFO_DEPTH + 1);	
71			end	
72			else if (ifc.full) begin	
73	1	502	count <= (count - 1) % (FIFO_DEPTH + 1);	
74			end	
75			end	
76			end	
77			end	
78				
79	1	10156	assign ifc.full = (count == FIFO_DEPTH)? 1 : 0;	
80	1	10156	assign ifc.empty = (count == 0)? 1 : 0;	
81			// BUG underflow signal should be sequential.	
82	1	10156	assign ifc.almostfull = (count == FIFO_DEPTH-1)? 1 : 0; //==> bug almost full should assert when the fifo has only one empty space	
83	1	10156	assign ifc.almostempty = (count == 1)? 1 : 0;	
84			//===== ASSERTATION =====	
85			//-->RESET_CHK	
86			#ifdef SIM	
87	1	14928	always_comb begin : RESET_CHK	

-Top module:-

```
module top ();
    bit clk;
    //----- clock generation -----
    initial begin
        clk=0;
        forever begin
            #5 clk= ~clk;
        end
    end
    //----- modules instantiations -----
    FIFO_ifc    ifc(clk);
    FIFO        DUT(ifc);
    FIFO_tb     tb (ifc);
    FIFO_monitor monitor(ifc);
    //----- reset asseration -----
    always_comb begin
        if(! ifc.rst_n)
            RESET_OUT_CHK: assert final(ifc.data_out == 0)
        else    $error("ERROR IN RESETING THE DATA_OUT asychronusly");
    end
endmodule
```

-Monitor module:-

```
import shared_pkg::*;
import FIFO_cov_pkg::*;
import FIFO_trans_pkg::*;
import FIFO_scoreboard_pkg::*;

module FIFO_monitor (FIFO_ifc.monitor ifc);
    FIFO_transaction signals;
    FIFO_cov          cov_collect;
    FIFO_scoreboard   scoreboard;
    initial begin
        signals      = new();
        cov_collect  = new();
        scoreboard   = new();
        forever begin
            wait(samp.triggered);
```



```

@(negedge ifc.clk);
signals.rst_n      = ifc.rst_n;
signals.wr_en      = ifc.wr_en;
signals.rd_en      = ifc.rd_en;
signals.data_in    = ifc.data_in;
signals.data_out   = ifc.data_out;
signals.almostfull = ifc.almostfull;
signals.empty      = ifc.empty;
signals.almostempty = ifc.almostempty;
signals.overflow   = ifc.overflow;
signals.full       = ifc.full;
signals.underflow  = ifc.underflow;
signals.wr_ack     = ifc.wr_ack;

fork
//----- coverage thread -----
begin
cov_collect.sample_data(signals);
end
//----- correctness checking thread -----
begin
scoreboard.check_result(signals);
end
join
if (test_finished) begin
@(posedge ifc.clk);
$display("=====> TEST FINISHED
<=====");
$display("correct = %0d , errors = %0d", correct_count , error_count);
$stop;
end
end
end
endmodule

```

-testbench module:-

```
import FIFO_trans_pkg::*;
import shared_pkg::*;
module FIFO_tb (FIFO_ifc.tb ifc);
    FIFO_transaction packet;
    //----- stimulus generation -----
    initial begin
        packet = new(30,70);
        ifc.rst_n = 0;
        -> samp;
        @(negedge ifc.clk);
        ifc.rst_n = 1;
        -> samp;
        @(negedge ifc.clk);
        repeat(10000) begin
            assert (packet.randomize())
            else $fatal("packet can't be randomized");
            ifc.rst_n = packet.rst_n;
            ifc.wr_en = packet.wr_en;
            ifc.rd_en = packet.rd_en;
            ifc.data_in = packet.data_in;
            -> samp;
            @(negedge ifc.clk);
        end
        packet = new(80,40);
        repeat(10000) begin
            assert (packet.randomize())
            else $fatal("packet can't be randomized");
            ifc.rst_n = packet.rst_n;
            ifc.wr_en = packet.wr_en;
            ifc.rd_en = packet.rd_en;
            ifc.data_in = packet.data_in;
            -> samp;
            @(negedge ifc.clk);
        end
        test_finished = 1;
        ->samp;
    end
endmodule
```

-Scoreboard package:-

```
package FIFO_scoreboard_pkg;
import FIFO_trans_pkg::*;
import shared_pkg::*;
class FIFO_scoreboard;
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
//----- VARIABLES -----
logic [FIFO_WIDTH-1:0] data_out_ref = 0;
logic wr_ack_ref = 0, overflow_ref = 0;
logic full_ref = 0, empty_ref = 0;
logic almostfull_ref = 0, almostempty_ref = 0;
logic underflow_ref = 0;
logic [FIFO_WIDTH-1:0] mem_ref[$:FIFO_DEPTH];

//===== METHODS =====
function void reference_model(FIFO_transaction F_txn);
    if (!F_txn.rst_n) begin
        mem_ref.delete();
        data_out_ref = 0;
    end
    else begin
        //----- write operation -----
        full_ref = (mem_ref.size() == FIFO_DEPTH) ? 1:0;
        almostfull_ref = (mem_ref.size() == FIFO_DEPTH - 1) ? 1:0;
        empty_ref = (mem_ref.size() == 0) ? 1:0;
        almostempty_ref = (mem_ref.size() == 1) ? 1:0;
        if (F_txn.wr_en) begin
            if (!full_ref) begin
                mem_ref.push_back(F_txn.data_in);
                wr_ack_ref = 1;
                overflow_ref = 0;
            end
            else begin
                overflow_ref = 1;
                wr_ack_ref = 0;
            end
        end
        //----- read operation -----
        if (F_txn.rd_en) begin
            if (!empty_ref) begin
                data_out_ref = mem_ref.pop_front();
                underflow_ref = 0;
            end
        end
    end
end
```

```

        else begin
            data_out_ref = 0;
            underflow_ref = 1;
        end
    end
    //----- flags assignments -----
end
endfunction

function void check_result(FIFO_transaction F_txn);
    reference_model(F_txn);
    if(F_txn.data_out == data_out_ref) begin
        correct_count ++;
    end
    else begin
        $error("data_out = %0d, expected = %0d" , F_txn.data_out, data_out_ref);
        error_count ++;
    end
endfunction

endclass
endpackage

```

-Coverage package:-

```

package FIFO_cov_pkg;
import FIFO_trans_pkg::*;

class FIFO_cov;
    FIFO_transaction F_cvg_txn;

    //===== coverage =====
    covergroup cvg;
        //----- coverpoints -----
        WR_EN_COV:coverpoint F_cvg_txn.wr_en{
            option.weight =0;
            bins wr_high = {1'b1} ;
            bins wr_low  = {1'b0} ;
        }
        RD_EN_COV: coverpoint F_cvg_txn.rd_en{
            option.weight =0;
            bins rd_high = {1'b1} ;

```

```

    bins rd_low  = {1'b0} ;
}
FULL_COV: coverpoint F_cvg_txn.full{
    option.weight =0;
    bins full_high = {1'b1} ;
    bins full_low  = {1'b0} ;
}
A_FULL_COV: coverpoint F_cvg_txn.almostfull{
    option.weight =0;
    bins a_full_high = {1'b1} ;
    bins a_full_low  = {1'b0} ;
}
EMPTY_COV: coverpoint F_cvg_txn.empty{
    option.weight =0;
    bins empty_high = {1'b1} ;
    bins empty_low  = {1'b0} ;
}
A_EMPTY_COV: coverpoint F_cvg_txn.almostempty{
    option.weight =0;
    bins a_empty_high = {1'b1} ;
    bins a_empty_low  = {1'b0} ;
}
}
OVERFLOW_COV: coverpoint F_cvg_txn.overflow{
    option.weight =0;
    bins overflow_high = {1'b1} ;
    bins overflow_low  = {1'b0} ;
}
UNDERFLOW_COV: coverpoint F_cvg_txn.underflow{
    option.weight =0;
    bins under_high = {1'b1} ;
    bins under_low  = {1'b0} ;
}
}
WR_ACK_COV: coverpoint F_cvg_txn.wr_ack{
    option.weight =0;
    bins wr_ack_high = {1'b1} ;
    bins wr_ack_low  = {1'b0} ;
}
}
//----- CROSS COVERAGE -----
FULL_CROSS:      cross WR_EN_COV, RD_EN_COV, FULL_COV {
    ignore_bins cant_occur = binsof(RD_EN_COV.rd_high) &&
binsof(FULL_COV.full_high);
}
A_FULL_CROSS:      cross WR_EN_COV, RD_EN_COV, A_FULL_COV;
A_EMPTY_CROSS:      cross WR_EN_COV, RD_EN_COV, A_EMPTY_COV;
EMPTY_CROSS:        cross WR_EN_COV, RD_EN_COV, EMPTY_COV;

```

```

OVERFLOW_CROSS:   cross WR_EN_COV, RD_EN_COV, OVERFLOW_COV;
UNDERFLOW_CROSS: cross WR_EN_COV, RD_EN_COV, UNDERFLOW_COV;
WR_ACK_CROSS:     cross WR_EN_COV, RD_EN_COV, WR_ACK_COV;
endgroup

//===== methods =====

function new();
    cvg = new();
endfunction

function void sample_data(FIFO_transaction F_txn);
    F_cvg_txn = F_txn;
    cvg.sample();
endfunction
endclass //FIFO_cov
endpackage

```

-Transaction package:-

```

package FIFO_trans_pkg;
class FIFO_transaction;
    parameter          FIFO_WIDTH = 16;
    parameter          FIFO_DEPTH = 8;
    int                 RD_EN_ON_DIST, WR_EN_ON_DIST;
    rand bit            rst_n, wr_en, rd_en;
    rand bit [FIFO_WIDTH-1:0] data_in;
    logic [FIFO_WIDTH-1:0] data_out;
    logic               wr_ack, overflow;
    logic               full, empty, almostfull, almostempty, underflow;

    //===== constructor =====
    function new(int RD_VAL=30, int WR_VAL=70);
        RD_EN_ON_DIST = RD_VAL;
        WR_EN_ON_DIST = WR_VAL;
    endfunction

    //===== constraints definition =====

    //----> RST_CHK
    constraint rst_n_c {
        rst_n dist {1 := 95, 0 := 5};
    }

```

```

//-----> WR_EN_CHK
constraint wr_en_c {
    wr_en dist {1 := WR_EN_ON_DIST , 0 := 100-WR_EN_ON_DIST};
}

//-----> RD_EN_CHK
constraint rd_en_c {
    rd_en dist {1 := RD_EN_ON_DIST , 0 := 100 - RD_EN_ON_DIST};
}
endclass

endpackage

```

-FIFO interface:-

```

interface FIFO_ifc(input bit clk);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;
logic                rst_n, wr_en, rd_en;
logic [FIFO_WIDTH-1:0] data_in;
logic [FIFO_WIDTH-1:0] data_out;
logic                wr_ack, overflow;
logic                full, empty, almostfull, almostempty, underflow;

modport dut (
    input    clk,
            rst_n,
            wr_en,
            rd_en,
            data_in,

    output   wr_ack,
            data_out,
            full,
            empty,
            almostfull,
            almostempty,
            overflow,
            underflow
);

```

```
modport tb (  
    output  clk,  
           rst_n,  
           wr_en,  
           rd_en,  
           data_in,  
  
    input   wr_ack,  
           data_out,  
           full,  
           empty,  
           almostfull,  
           almostempty,  
           overflow,  
           underflow  
);  
  
modport monitor (  
    input  clk,  
          rst_n,  
          wr_en,  
          rd_en,  
          data_in,  
          wr_ack,  
          data_out,  
          full,  
          empty,  
          almostfull,  
          almostempty,  
          overflow,  
          underflow  
);  
  
endinterface
```

-FIFO Design:-

```
////////////////////////////////////
// Author: Kareem Waseem
// Course: Digital Verification using SV & UVM
//
// Description: FIFO Design
//
////////////////////////////////////
module FIFO(FIFO_ifc.dut ifc);
parameter FIFO_WIDTH = 16;
parameter FIFO_DEPTH = 8;

localparam max_fifo_addr = $clog2(FIFO_DEPTH);

reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];

reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
reg [max_fifo_addr:0] count; //bug count didn't wraparound

always @(posedge ifc.clk or negedge ifc.rst_n) begin
    if (!ifc.rst_n) begin

        wr_ptr    <= 0;

    end
    else if (ifc.wr_en && count < FIFO_DEPTH) begin
        mem[wr_ptr] <= ifc.data_in;
        ifc.wr_ack <= 1;
        wr_ptr <= wr_ptr + 1;
    end
    else begin
        ifc.wr_ack <= 0;
        if (ifc.full & ifc.wr_en)
            ifc.overflow <= 1;
        else
            ifc.overflow <= 0;
        end
    end
end

always @(posedge ifc.clk or negedge ifc.rst_n) begin
    if (!ifc.rst_n) begin
        rd_ptr <= 0;
        // data_out signals didn't reset
        ifc.data_out <= 0;
    end
end
```

```

end
else
    if (ifc.rd_en && count != 0) begin
        ifc.data_out <= mem[rd_ptr];
        rd_ptr <= rd_ptr + 1;
        ifc.underflow <= 0; // bug
    end
    else if (ifc.rd_en && count == 0) begin
        ifc.data_out <= 0;
        ifc.underflow <= 1; // bug
    end
end

end

always @(posedge ifc.clk or negedge ifc.rst_n) begin
    if (!ifc.rst_n) begin
        count <= 0;
    end
    else begin
        if ( ({ifc.wr_en, ifc.rd_en} == 2'b10) && !ifc.full)
            count <= (count + 1);
        else if ( ({ifc.wr_en, ifc.rd_en} == 2'b01) && !ifc.empty)
            count <= (count - 1);
        //error didn't handle both writing and reading case
        else if ( ({ifc.wr_en, ifc.rd_en} == 2'b11)) begin
            if (ifc.empty) begin
                count <= (count + 1);
            end
            else if (ifc.full) begin
                count <= (count - 1);
            end
        end
    end
end

end

end

assign ifc.full = (count == FIFO_DEPTH)? 1 : 0;
assign ifc.empty = (count == 0)? 1 : 0;
// BUG underflow signal should be sequential.
assign ifc.almostfull = (count == FIFO_DEPTH-1)? 1 : 0; //==> bug almost full
should assert when the fifo has only one empty space
assign ifc.almostempty = (count == 1)? 1 : 0;
//===== ASSERTION =====
//-->RESET CHK
`ifdef SIM
    always_comb begin : RESET_CHK

```

```

    if(!ifc.rst_n) begin
        ASNYC_RST: assert final( wr_ptr == 0  && rd_ptr == 0 && count == 0 )
        else $error("error in resetting the internal pointers and counters");
    end
end

//-->WR_ACK_CHK
property wr_ack_p;
@(posedge ifc.clk) disable iff(!ifc.rst_n)
    (ifc.wr_en && !ifc.full) |=> ifc.wr_ack;
endproperty
WR_ACK_CHK:assert property (wr_ack_p)
else $error("error in writing ack");
cover property (wr_ack_p);

//--> OVERFLOW_CHK
property overflow_p;
@(posedge ifc.clk) disable iff(!ifc.rst_n)
    (ifc.wr_en && ifc.full) |=> ifc.overflow ;
endproperty
OVERFLOW_CHK:assert property (overflow_p)
else $error("error in overflow flag");
cover property (overflow_p);

//--> UNDERFLOW_CHK
property underflow_p;
@(posedge ifc.clk) disable iff(!ifc.rst_n)
    (ifc.rd_en && ifc.empty) |=> ifc.underflow ;
endproperty
UNDERFLOW_CHK:assert property (underflow_p)
else $error("error in underflow flag");
cover property (underflow_p);

//--> FULL_CHK
property full_p;
@(posedge ifc.clk)
    (count == FIFO_DEPTH) |-> ifc.full;
endproperty
FULL_CHK: assert property (full_p)
else $error("error in full flag");
cover property (full_p);

//--> EMPTY_CHK
property empty_p;
@(posedge ifc.clk)
    (count == 0) |-> ifc.empty;
endproperty

```

```

EMPTY_CHK: assert property (empty_p)
else $error("error in empty flag");
cover property (empty_p);

//--> ALMOSTFULL_CHK
property a_full_p;
  @(posedge ifc.clk)
    (count == FIFO_DEPTH - 1) |-> ifc.almostfull;
endproperty
ALMOST_FULL_CHK: assert property (a_full_p)
else $error("error in almostfull flag");
cover property (a_full_p);

//--> ALMOSTEMPTY_CHK
property a_empty_p;
  @(posedge ifc.clk)
    (count == 1) |-> ifc.almostempty;
endproperty
ALMOST_EMPTY_CHK: assert property (a_empty_p)
else $error("error in almostempty flag");
cover property (a_empty_p);

//--> write_pointer_wraparound
property wr_wrap_p;
  @(posedge ifc.clk) disable iff(!ifc.rst_n)
    ifc.wr_en && !ifc.full && wr_ptr == (FIFO_DEPTH - 1'b1) |=> (wr_ptr == 0);
endproperty
WR_WRAP: assert property (wr_wrap_p)
else $error("ERROR in wr_pointer wraparound");
cover property(wr_wrap_p);

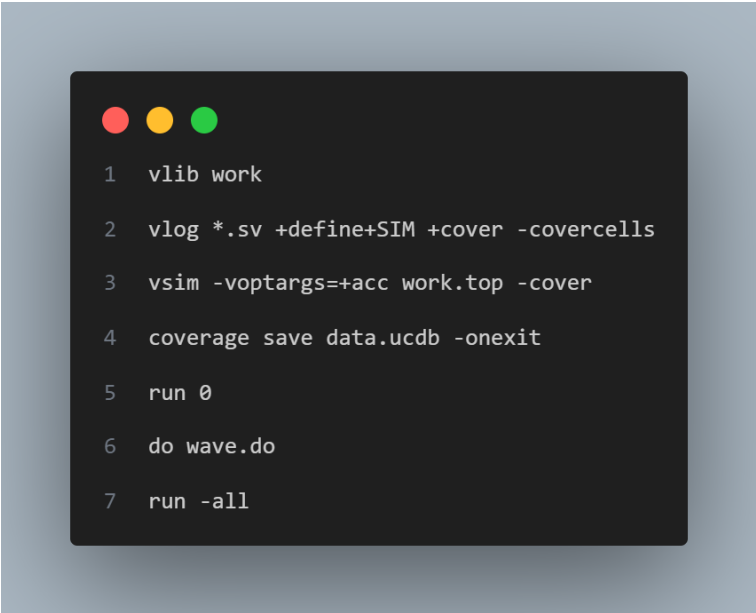
//-->READ_POINTER_WRAPAROUND
property rd_wrap_p;
  @(posedge ifc.clk) disable iff(!ifc.rst_n)
    ifc.rd_en && !ifc.empty && rd_ptr == (FIFO_DEPTH - 1'b1 ) |=> (rd_ptr == 0);
endproperty
RD_WRAP: assert property (rd_wrap_p)
else $error("ERROR in rd_pointer wraparound");
cover property(rd_wrap_p);

//--> POINTERS_THRESHOLD
property pointer_threshold;
  @(posedge ifc.clk)
    wr_ptr <= FIFO_DEPTH && rd_ptr <= FIFO_DEPTH && count <= FIFO_DEPTH;
endproperty

```

```
THRESHOLD:assert property(pointer_threshold)
else $error("POINTERS exceeded the FIFO_DEPTH");
cover property(pointer_threshold);
`endif
endmodule
```

-DO file:-



```
1 vlib work
2 vlog *.sv +define+SIM +cover -covercells
3 vsim -voptargs=+acc work.top -cover
4 coverage save data.ucdb -onexit
5 run 0
6 do wave.do
7 run -all
```

-Waveform:-

